Introduction to NLP

Natural Language Processing

Beginner Friendly

Hands-On Demo

spaCy · sklearn · HuggingFace

Mr. Shridhar Galande

Agenda

Part 1 Foundations of NLP

Part 2 Models & Applications

Session Agenda — Part 1

Topic	What You'll Learn
Introduction to NLP	What NLP is, real-world uses, pipeline overview
NLP Pipeline & Language Understanding	Lexical, Syntactic, Semantic, Pragmatic analysis
Text Preprocessing Basics	Tokenization, stemming, lemmatization, noise removal
Text Representation Techniques	BoW, N-grams, TF-IDF, Word Embeddings
Demo 1 — Text Preprocessing	Full spaCy pipeline on real movie reviews

What is NLP?

Teaching computers to understand human language

NLP (Natural Language Processing) is a branch of AI that helps machines read, understand, and generate text — just like a human would.

Google Search

Understands your query even with typos

Chatbots

ChatGPT, Claude, Gemini etc. understand & respond

Translation

Google Translate converts language instantly

Spam Filters

Gmail detects and blocks spam emails

💡 Key Insight: Computers see text as raw characters — NLP gives them the ability to understand meaning, intent, and context.

The NLP Pipeline

Text goes through multiple stages before a machine can understand it

```
Input: "The movie was absolutely fantastic! I loved every scene."
```

Each step builds on the previous one — like a factory assembly line for language!

Lexical Analysis

POS Tagging & Morphology — understanding individual words

"What are the words and what form are they in?" **->** The focus is on **tokenization**, **morphology**, and **word-level processing**.

What is it?

POS Tagging labels each word with its grammatical role:

Noun, Verb, Adjective, Adverb, etc.

Morphology studies word forms:

'run' → runs, running, ran

```
Sentence: "She runs fast in the park"
She → PRON (Pronoun)
runs → VERB
fast → ADV (Adverb)
in → ADP (Preposition)
the → DET (Determiner)
park → NOUN
```

Why does this matter?

Search Engines

Know that 'apple' is a noun (company or fruit?) from surrounding context

Grammar Check

Grammarly detects 'She run fast' is wrong because run must be VERB+3rd-person-s

Chatbots

Extract 'book a flight' → VERB=book, NOUN=flight → triggers booking flow

Syntactic Analysis

Dependency Parsing · Parse Trees · Chunking

Focuses on grammatical structure by analyzing how words combine into phrases and sentences, with key tasks such as Constituency Parsing, which identifies nested phrase structures like Noun Phrases (NP), Verb Phrases (VP), and Prepositional Phrases (PP).

Dependency Parsing

Maps word-to-word grammatical relations

Info extraction · Relation detection · Semantic search

Parse Trees

Hierarchical phrase structure grammar

Grammar checking · MT parsing · Syntax education

Chunking

Flat phrase grouping & extraction

NER grouping · Search indexing · QA span finding

Dependency Parsing & Parse Trees

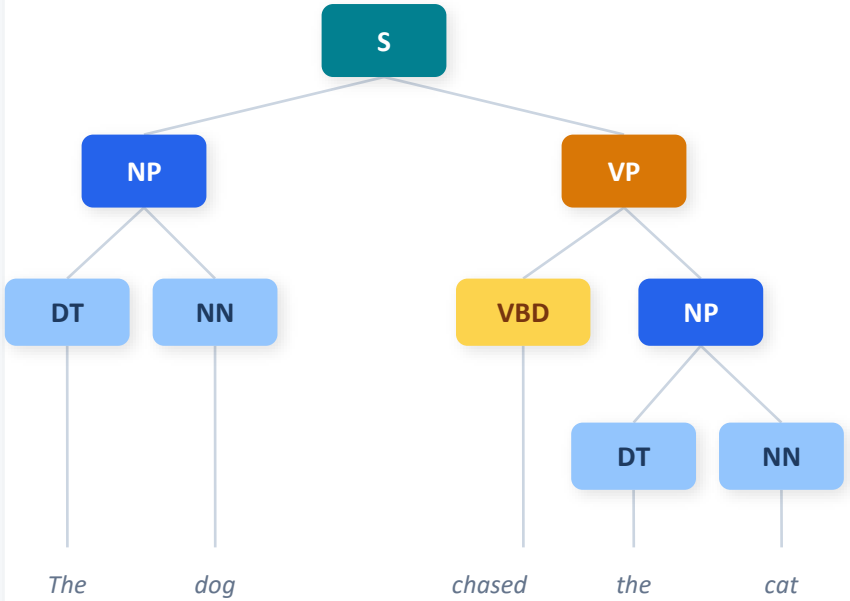
Dependency Parsing

"The dog chased the cat"

- ROOT → chased
- nsubj → dog
- dobj → cat
- det → The / the

Key: Arrows flow from head → dependent. Every word connects back to a single ROOT verb.

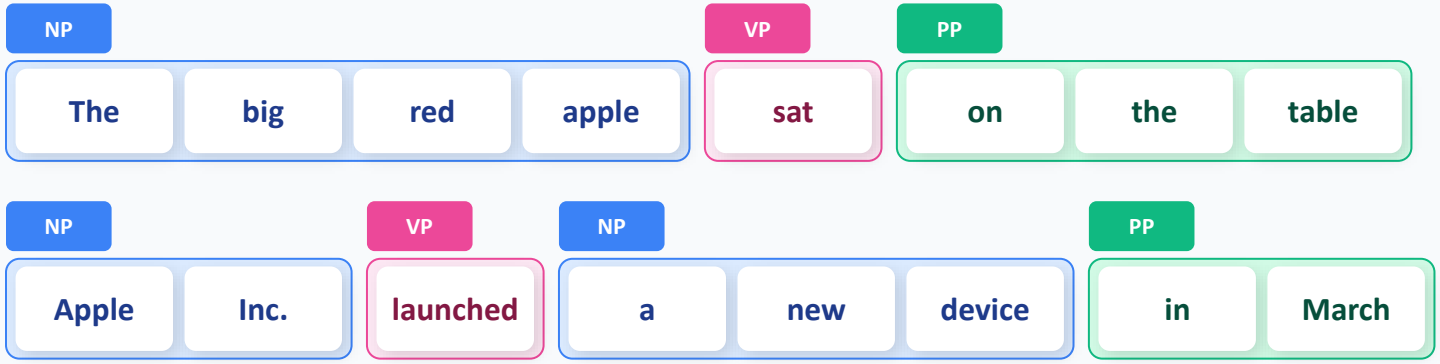
Parse Trees



Key: Phrases nest recursively — VP contains its own inner NP.

Chunking & Phrase Extraction

Chunking groups adjacent words into non-overlapping, non-recursive phrases (chunks). Unlike full parse trees, chunks are flat — faster and sufficient for most NLP pipelines.



Parse Tree vs. Chunking — key difference

	Parse Tree	Chunking
Depth	Fully recursive (VP → NP → DT + NN)	Flat, non-recursive brackets
Speed	Slower — full grammar required	Fast — regex / ML over POS tags
Use	MT, deep semantics, grammar checks	NER, search indexing, QA spans

Semantic & Pragmatic Analysis

Understanding meaning and real-world intent

NLP goes beyond grammar — it must understand what words mean (semantics) and what speakers intend (pragmatics). These two layers are essential for chatbots, sentiment analysis, machine translation, and dialogue systems.

Semantic Analysis

Meaning of words & sentences

- Word Sense Disambiguation
- Semantic Roles (Agent/Patient/Action)
- Context-dependent interpretation

Pragmatic Analysis

Intent behind language in context

- Sarcasm & irony detection
- Conversational implicature
- Speech acts & real-world requests

Semantic Analysis

Studies the MEANING of words and sentences — going beyond surface form to understand what is actually being communicated.

Word Sense Disambiguation (WSD)

"I went to the bank"

River bank

A slope of land beside a body of water

Financial bank

A financial institution for deposits

Context decides:

"fishing by the bank" → river; "deposit at the bank" → financial

Semantic Roles (Thematic Roles)

"John kicked the ball"

Agent **"John"** — Does the action

Patient **"ball"** — Receives the action

Action **"kicked"** — The event itself

Used in: machine translation, QA systems, event extraction

Pragmatic Analysis

Studies language in REAL-WORLD CONTEXT — the intent behind words. Goes beyond literal meaning to capture what the speaker actually means.

Sarcasm & Irony Detection

"Oh great, another Monday!"

Literal "Oh great" = positive words

Intent Actually expressing frustration / dread

Cues pragmatics uses:

Tone contrast · Hyperbole · Contextual incongruity · Prior discourse

Conversational Context & Speech Acts

"Can you pass the salt?"

Literal A question about physical ability

Intent A polite REQUEST to pass the salt

NOT Asking if you are physically capable

Grice's Maxims — why pragmatics works:

Quantity · Quality · Relation · Manner

Text Preprocessing Basics

Cleaning and preparing raw text before feeding it to any model

🍕 **Analogy:** Before cooking, you wash vegetables, peel them, and chop them up. Preprocessing does the same for text — we clean and normalize it before the 'cooking' (model training) begins.

✂️ Sentence Segmentation

```
"Hello world. How are you?" → ["Hello world.", "How are you?"]
```

🔤 Tokenization

```
"I love NLP!" → ["I", "love", "NLP", "!"]
```

🧹 Noise Removal & Regex

```
"Visit http://nlp.ai !!! 😊" → "Visit"
```

🔡 Lowercasing

```
"Apple" and "apple" → same token "apple"
```

🛑 Stopword Removal

```
"the cat sat on the mat" → "cat sat mat"
```

🌱 Stemming

```
"running", "runs", "runner" → "run"
```

Stemming vs Lemmatization

Both reduce words to their base form — but very differently

Why reduce words? "running", "runs", "ran" all mean the same thing. Reducing them to a base form helps machines treat them as one concept — critical for search, classification, and text mining.

Stemming (Rough & Fast)

Chops off word endings using simple rules. No dictionary — it just strips suffixes.

Output may NOT be a real word!

Examples (Porter Stemmer):

`running → run`	✓
`studies → studi`	⚠
`better → better`	✗ no change!
`historical → histor`	⚠

Best for: speed-critical tasks — search indexing, large corpora

Lemmatization (Smart & Accurate)

Uses a full vocabulary dictionary and grammar rules. Always returns a real, valid word.

Output is ALWAYS a valid dictionary word!

Examples (spaCy Lemmatizer):

`running → run`	✓
`studies → study`	✓
`better → good`	✓
`historical → historical`	✓

Best for: sentiment analysis, QA systems, chatbots — accuracy matters

Stemming vs Lemmatization

Quick comparison & when to use each

	Stemming	Lemmatization
Method	Suffix-stripping rules	Dictionary + grammar rules
Speed	Very fast	Slower (dictionary lookup)
Accuracy	Lower — may get nonsense	High — always real words
Language	Language-specific rules	Needs trained vocabulary
Output	"studi", "histor"	"study", "historical"
Use case	Search engines, indexing	NLU, sentiment, QA, chatbots

Choose Stemming when: millions of docs, speed is critical, slight inaccuracy is OK

Choose Lemmatization when: meaning matters — chatbots, NLU pipelines, QA

Text Representation — Bag of Words (BoW)

How do we turn words into numbers machines can process?

Core Idea: Count how many times each word appears in a document. Ignore word order — treat the document as a literal "bag" of words.

Step-by-step example

1 Doc 1: "cat sat on mat"

2 Doc 2: "cat ate the rat"

Build vocabulary (unique words):

cat	sat	on	mat	ate	the	rat

BoW Vectors:

Doc1:	[	1	1	1	1	0	0	0]
Doc2:	[	1	0	0	0	1	1	1]

Strengths

✓ Simple and fast to compute

✓ Works well for short texts

✓ Great baseline for text classification

✓ Easy to implement in any language

Limitations

✗ Loses all word order — "not good" = "good"?

✗ Vocabulary size explodes with large corpora

✗ No concept of similar words ("happy" ≠ "joyful")

✗ Sparse vectors — mostly zeros

N-grams — Capturing Context

Groups of N consecutive words — preserving word order locally

The problem with BoW: "New York" gets split into "New" + "York" — losing the meaning. "not good" becomes just "good". N-grams fix this by grouping adjacent words.

```
Sentence:  "I  love  NLP  very  much"
```

Unigrams (1-gram) *Same as BoW — individual words*

`"I"` `"love"` `"NLP"` `"very"` `"much"`

Bigrams (2-gram) *Pairs of words — captures phrases*

`"I love"` `"love NLP"` `"NLP very"` `"very much"`

Trigrams (3-gram) *Triplets — richer context*

`"I love NLP"` `"love NLP very"` `"NLP very much"`

Rule of thumb: N=1 for broad coverage · N=2 for phrases · N=3+ for specific patterns. Higher N = more context but sparser data.

TF-IDF — The Formula

Term Frequency × Inverse Document Frequency

Intuition: A word is important if it appears OFTEN in THIS document, but RARELY across ALL documents. "the" appears everywhere → low importance. "photosynthesis" is rare → high importance in biology docs.

TF — Term Frequency

How often does the word appear in THIS document?

$$TF(t, d) = \frac{\text{Number of times term t appears in doc d}}{\text{Total number of words in doc d}}$$

IDF — Inverse Document Frequency

How RARE is this word across ALL documents?

$$IDF(t) = \log\left(\frac{\text{Total number of documents}}{\text{Documents containing term t}}\right)$$

$$\text{TF-IDF}(t, d) = TF(t, d) \times IDF(t)$$

TF-IDF — Worked Example

Seeing the formula in action with a film review document

Document: "The movie had brilliant cinematography and the acting was superb." | Corpus: 1000 movie reviews

Word	TF (in doc)	IDF (across docs)	TF-IDF score	Importance
the	0.12	0.01 (very common)	0.001	Low
movie	0.05	1.20 (somewhat rare)	0.060	Medium
cinematography	0.02	3.80 (very rare)	0.076	High
acting	0.04	1.50 (somewhat rare)	0.060	Medium
superb	0.01	2.90 (rare)	0.029	Medium

Key insight: "the" has high TF (appears often) but near-zero IDF (appears in ALL docs) → TF-IDF ≈ 0. "cinematography" has low TF but high IDF → TF-IDF is highest. TF-IDF automatically finds the DISTINCTIVE words.

Word Embeddings — Intuition

Representing words as vectors in a meaningful space

Problem with BoW / TF-IDF

✗ "king" and "queen" treated as completely unrelated

✗ "happy" and "joyful" look totally different to the model

✗ No understanding of word similarity

✗ Cannot handle unseen words ("OOV problem")

Word Embeddings Solution

Each word is mapped to a dense vector of numbers. Similar words end up CLOSE in vector space!

```
king    →  [0.2,  0.8,  0.1, ...]
queen   →  [0.2,  0.7,  0.2, ...]
happy   →  [0.9,  0.1,  0.7, ...]
joyful  →  [0.8,  0.1,  0.7, ...]  ← very close!
```

Famous Word2Vec result: "king" – "man" + "woman" ≈ "queen"

Word2Vec

Predicts surrounding context words (CBOW) or target from context (Skip-gram). Most famous embedding model.

Google · 2013

GloVe

Uses global word co-occurrence statistics from large corpora. Combines local context with global matrix factorization.

Stanford · 2014

FastText

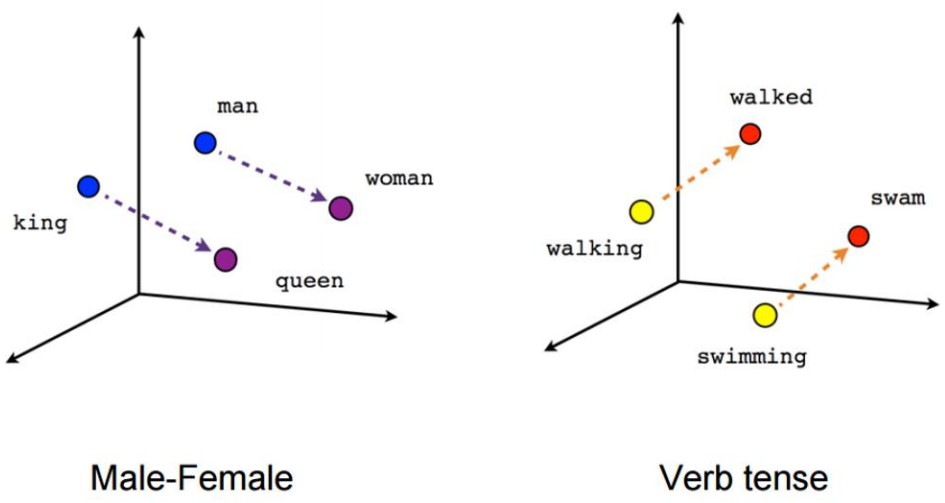
Breaks words into character n-grams. Handles unseen words, misspellings, and morphologically rich languages.

Facebook · 2016

Word Embeddings — Vector Space

How words cluster by meaning in high-dimensional space

Semantic clusters in vector space



Key properties

Similarity = distance
Cosine similarity measures how close two word vectors are

Arithmetic works
"king" – "man" + "woman" = "queen" (vector math!)

Dimensions = features
Each dimension may encode gender, age, royalty, etc.

Typically 100–300 dims
Dense vectors vs sparse BoW vectors of 100,000+

Transfer learning
Pre-trained on billions of words, reuse in your task

Demo 1 — Pre-processing + Linguistic Analysis

IMDb Movie Reviews · spaCy Pipeline

`Input: "The Shawshank Redemption was an absolutely breathtaking film!" → Output: Tokens + POS Tags + Chunks + Lemmas`

Break Time!

10 minutes

Part 2 — Models & Applications

From raw text to real predictions

Machine Learning · Deep Learning · Transformers

ML for NLP

Common NLP Tasks

Deep Learning & LLMs

Demo 2 — 3 Ways

Summary & Q&A

Machine Learning for NLP

Teaching a model to learn patterns from labeled text examples

🎓 **How it works:** We convert text to numbers (features), then give a model thousands of labeled examples (e.g., reviews marked positive/negative). The model finds patterns that separate the classes — then predicts on new, unseen text.

Step 1

Feature Engineering from Text

Convert cleaned text to a numeric matrix using TF-IDF or BoW.

Each row = 1 document

Each column = 1 word/n-gram

Step 2

Training Data Preparation

Split data into:

- Train set (80%) — model learns
- Test set (20%) — we evaluate

Label encode: positive=1, negative=0

Step 3

Train the Classifier

Feed features + labels → model learns decision boundary.

Naive Bayes: fast, probabilistic

Logistic Regression: robust, interpretable

Naive Bayes for Text Classification

A probabilistic model — asks 'how likely is this text to belong to each class?'

💡 **Intuition:** If the word "amazing" appears in 90% of positive reviews and only 5% of negative ones, seeing "amazing" in a new review is strong evidence it's positive. Naive Bayes multiplies these probabilities for every word.

📊 Worked Example

New review: "The film was fantastic!“

```
P(POSITIVE | "The film was fantastic!")
∝  P(POSITIVE) × L("The"|POS) ×
L("film"|POS) × L("was"|POS) ×
L("fantastic"|POS)
```

```
P(NEGATIVE | "The film was fantastic!") ∝
P(NEGATIVE) × L("The"|NEG) × L("film"|NEG)
× L("was"|NEG) × L("fantastic"|NEG)
```

→ **Model predicts: POSITIVE** ✅

Why 'Naive'?

It assumes each word is independent of others — not always true, but works surprisingly well in practice!

💻 sklearn Code

```
from sklearn.feature_extraction.text \
    import TfidfVectorizer
from sklearn.naive_bayes \
    import MultinomialNB
from sklearn.pipeline import Pipeline

# Build pipeline
model = Pipeline([
  ('tfidf', TfidfVectorizer()),
  ('clf',   MultinomialNB())
])

# Train
model.fit(X_train, y_train)

# Evaluate
acc = model.score(X_test, y_test)
print(f"Accuracy: {acc:.2%}")
# → Accuracy: 85.3%
```

Logistic Regression for NLP

Learns a weighted score for each word — more interpretable than you'd expect

⚖️ **Intuition:** Each word gets a weight. "excellent" → +2.3, "boring" → −1.8, "the" → ~0. The model sums these weights for all words in the review, applies a sigmoid function, and outputs a probability between 0 and 1.

🔍 Learned Word Weights (IMDb)

Word	Weight	Effect
excellent	+2.41	↑ Positive
masterpiece	+2.18	↑ Positive
waste	−2.05	↓ Negative
awful	−1.97	↓ Negative
the	+0.01	≈ Neutral
film	+0.12	≈ Neutral

💻 sklearn Code

```
from sklearn.linear_model \
    import LogisticRegression

model = Pipeline([
  ('tfidf', TfidfVectorizer(
      ngram_range=(1, 2),
      max_features=50000
  )),
  ('clf', LogisticRegression(
      max_iter=1000
  ))
])

model.fit(X_train, y_train)
print(model.score(X_test, y_test))
# → Accuracy: 89.1%

# Bonus: inspect learned weights!
feat_names = model.named_steps['tfidf'].get_feature_names_out()
coefs      = model.named_steps['clf'].coef_[0]
```

Common NLP Tasks

What can NLP actually do? — Real tasks with real-world examples

😊😐😞 Sentiment Analysis

```
Input: "This movie blew my mind!"
Output: POSITIVE (confidence: 97%)

Used in: product reviews, brand monitoring, social
media analytics, customer feedback
```

🏷️ Named Entity Recognition (NER)

```
"Elon Musk founded SpaceX in California in 2002"

PERSON → Elon Musk
ORG → SpaceX
LOC → California
DATE → 2002
```

📝 Summarization

```
Input: [500-word news article about climate
change]
Output: "Scientists warn global temperatures will
rise 1.5°C by 2035 unless emissions cut 40%."

Used in: news apps, legal docs, research
```

❓ Machine Translation & Q&A

```
Translation:
"Hello world" → "Bonjour le monde"

Question Answering:
Context: [Wikipedia article]
Q: "When was Python created?"
A: "1991 by Guido van Rossum"
```

Sentiment Analysis — Deep Dive

One of the most widely used NLP tasks in industry

Approach 1
Rule-Based

Manually curated word lists.

'amazing' -> +1, 'awful' -> −1.

Sum of scores.

✅ Simple

❌ Can't handle sarcasm, context

Approach 2
ML-Based

TF-IDF + Logistic Regression / Naive Bayes.

Train on labeled reviews.

~89% accuracy on IMDb.

✅ No manual rules

❌ Misses word order

Approach 3
Transformer

DistilBERT / RoBERTa fine-tuned.

Understands full sentence context.

~87%+ accuracy on IMDb.

✅ Best accuracy

❌ Slower, needs GPU

```
from transformers import pipeline
sentiment = pipeline("sentiment-analysis", model="distilbert-base-uncased-finetuned-sst-2-english")
print(sentiment("This movie was absolutely incredible!"))  # → [{"label": "POSITIVE", "score": 0.9998}]
```

Named Entity Recognition (NER)

Automatically finding and classifying key information in text

"Apple was founded by Steve Jobs in Cupertino, California in April 1976."

ORG

`"Apple"`

Organization / Company

PERSON

`"Steve Jobs"`

People's names

GPE

`"Cupertino, California"`

Geo-Political Entity (cities, countries)

DATE

`"April 1976"`

Dates, times, periods

```
import spacy
nlp = spacy.load("en_core_web_sm")
doc = nlp("Apple was founded by Steve Jobs in
Cupertino...")

for ent in doc.ents:
    print(ent.text, "→", ent.label_)
# Apple → ORG
# Steve Jobs → PERSON
# Cupertino → GPE
```

Real-World Uses:

- **Reuters:** auto-tag news by mentioned companies
- **Google Maps:** extract locations from text
- **Healthcare:** find drug names in clinical notes
- **Legal:** identify parties, dates in contracts

Deep Learning & Modern NLP

How neural networks revolutionized language understanding

Pre-2013
Classical NLP

Hand-crafted rules, BoW, TF-IDF, Naive Bayes. No understanding of word meaning.

2013–2017
Word Embeddings

Word2Vec, GloVe. Words now have meaning in vector space. RNNs & LSTMs for sequences.

2017
Transformers Born

"Attention Is All You Need" paper (Google). Attention mechanism replaces RNN — processes ALL words at once in parallel.

2018+
BERT / GPT Era

Pre-train on massive text, fine-tune on your task. One model for everything. 95%+ accuracy across tasks.

🔑 Key Innovation: "Attention" lets the model focus on relevant words regardless of distance. "The animal didn't cross the street because IT was too tired." — Attention knows IT = animal.

BERT vs GPT — Two Flavors of Transformers

Different training objectives → different strengths

🔍 BERT (Bidirectional)

Made by: Google (2018)

Training: Reads text LEFT ↔ RIGHT (both directions at once) — like filling blanks:
`"The [MASK] sat on the mat" → cat`

Best at: Understanding tasks
- Sentiment analysis
- NER
- Question Answering
- Text classification

Variants: DistilBERT (40% smaller, retains ~97% of BERT performance), RoBERTa, ALBERT

✍️ GPT (Autoregressive)

Made by: OpenAI (GPT-1 2018 → GPT-4 2023)

Training: Reads LEFT → only, predicts NEXT word:
`"Once upon a [?]" → time`

Best at: Generation tasks
- Writing, storytelling
- Code generation
- Chatbots (ChatGPT!)
- Summarization

Variants: GPT-2 (open), GPT-3/4 (API), LLaMA, Mistral, Claude

Large Language Models (LLMs)

Transformers trained at massive scale — the foundation of modern AI

📖 **Scale is the key:** GPT-3 has 175 billion parameters and was trained on 570GB of internet text. At this scale, emergent capabilities appear — models can reason, write code, and solve math problems they were never explicitly trained on.

📚

Training Data

Hundreds of billions of words from: Wikipedia, books, GitHub, Reddit, web crawls.

Learns grammar, facts, reasoning, code.

🔢

Parameters

BERT: 110M params

GPT-2: 1.5B params

GPT-3: 175B params

GPT-4: (not published)

More params = more capacity to learn.

🎯

Zero-Shot Tasks

LLMs can do tasks they were NEVER explicitly trained on:

-> Translate French

-> Write code

-> Solve logic puzzles

-> Just from the prompt!

⚡

Using via API

You don't need to train an LLM. Just call the API:

from openai import OpenAI

client.chat.completions .create(model='gpt-4'...)

or

HuggingFace pipeline()

🛠️ Demo 2.1 — Text Classification: 3 Ways

Same IMDb Dataset · Compare accuracy across approaches

① ML Approach

Logistic Regression + TF-IDF

1. Vectorize text with TfidfVectorizer
2. Train LogisticRegression
3. Predict + evaluate accuracy
4. Inspect most predictive words

~89% accuracy

Fast ⚡

② DL Approach

Simple LSTM

1. Tokenize + pad sequences
2. Embedding layer → LSTM → Dense
3. Train with binary crossentropy
4. Plot training/validation curves

~87–91% accuracy

Medium 🧠

③ Pretrained

HuggingFace DistilBERT

1. Load pipeline('sentiment-analysis')
2. Run on IMDb test reviews
3. Map labels → compute accuracy
4. Zero fine-tuning needed!

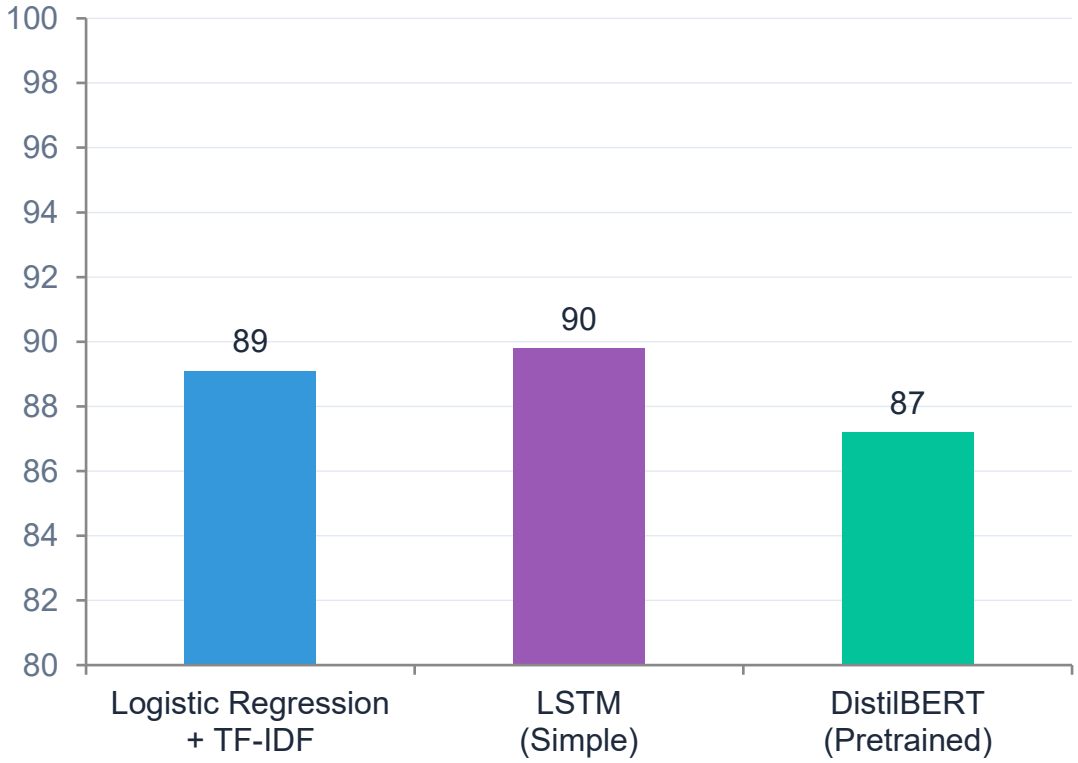
~87%+ accuracy

Best 🏆

without fine-tuning

Model Comparison — IMDb Sentiment

Accuracy vs. complexity tradeoff across all three approaches



Method	Accuracy	Train Time	GPU?
Logistic Reg.	89.1%	< 30 sec	No ✅
LSTM	89.8%	5–10 min	Helpful
DistilBERT	87.2%	Zero*	Yes ❌

** Zero training time: we use the pre-trained model directly, no fine-tuning*

💡 Right model depends on: accuracy needs, latency, infrastructure, and budget.

Summary — What You Learned Today

From raw text to production-ready NLP — in one session

Part 1 — Foundations

✅ NLP pipeline: tokenize → clean → tag → parse → understand

✅ Text preprocessing: stemming, lemmatization, stopwords, regex

✅ Text representation: BoW, N-grams, TF-IDF, Word Embeddings

✅ Lexical, Syntactic, Semantic & Pragmatic analysis

Part 2 — Models & Applications

✅ ML models: Naive Bayes & Logistic Regression for classification

✅ Common tasks: Sentiment, NER, Summarization, QA, Translation

✅ Deep Learning: Transformers, BERT vs GPT, the Attention mechanism

✅ 3 ways to classify text — and when to use each

🧭 **Mental Model:** Raw Text → Clean & Tokenize → Represent as Numbers → Feed to Model (ML / DL / Pretrained) → Predict & Evaluate

Recommended Resources & Next Steps

Where to go from here

📚 Learn More

- 📖 Speech & Language Processing
- 🎓 NLP Courses
  Practical deep learning
- 🤗 HuggingFace Course
  — huggingface.co/course
- 📺 Stanford CS224N
  — NLP with Deep Learning

🛠️ Practice Projects

- 🎬 Kaggle IMDb Dataset
  — Build your own classifier
- 📰 News Category Classifier
  — Multi-class NLP
- 🐦 Twitter Sentiment Analysis
  — Real streaming data
- 📄 Resume Parser with NER
  — Extract names, skills, dates

⚡ Next Topics

- 🔥 Fine-tuning BERT on
  custom datasets
- 🔗 RAG (Retrieval-Augmented
  Generation) with LLMs
- 🌐 Building NLP APIs with
  FastAPI + HuggingFace
- 📊 Topic Modeling with
  LDA & BERTopic

Demo2.2
Application of Transformers

Questions & Discussion

"Which of the 3 classification approaches would you use for your project, and why?"

"What NLP task would solve a real problem in your work or daily life?"

"What surprised you most about how machines understand text?"

Thank you for attending! 🎉 Notebooks & slides shared on portal.

Thank You